جامعة أم القرى
UMM AL-QURA UNIVERSITY

“Kababa" (Saudi Grilled Cuisine Restaurant)

Name: Hajer Sayah, Dana Bafaqih, Alaa Ajlan
ID: 445013975, 445013918, 445013943

Introduction to database
Instructor’s name: Arwa Halawani

Table of Contents:

1. **Conceptual and Logical Design**
   - 1.1 Scenario
   - 1.2 ER Diagram
   - 1.3 Relational Schema
2. **Implementation**
   - 2.1 Basic Select
   - 2.2 Aggregate Functions
   - 2.3 Join Between Tables
   - 2.4 Subqueries to Answer Question
   - 2.5 View

Roles and Responsibilities:

scenario	All students participated
ER Diagram	All students participated
Relational Schema.	All students participated
Create tables and Insert.	Dana (Employee, Orders_Status) Alaa (Category, Dish, Dish_Details) Hajer (Orders, Customer)
Select, Aggregate Functions, Joins, subqueries, View	Each students wrote two SQL queries

1. Conceptual and Logical Design.

1.2 Scenario

"**Kababa**" is a Saudi restaurant that specializes in grilled cuisine (mainly kebabs) and serves customers with dine-in, and takeaway.
The restaurant records its **Employees**, **Customers**, their **Orders**, and the **Dishes** included in each order, and the **Categories** to which these dishes belong.

- An **Employee** is identified by their ID, name, role (Waiter / Cashier / Chef), *Phone, Email,* and hire date.
- A **Customer** is identified by their ID, name, *Phone, and Email*.
- A **Dish** is identified by its ID, name, price, description, availability, preparation time, and calories. Each dish belongs to one Category (Main Dishes, Side Dishes, Drinks, and Sauces).
- A **Category** is identified by category ID, name (Main Dishes, Side Dishes, Drinks, and Sauces).
- An **Order** is identified by its ID, date, order type (Dine-in / Takeaway), order status (Pending / Preparing / Completed / Cancelled) (multi-valued), total amount, and payment method (Cash / Card).

Relationship:

1. **Customer – Places – Order**:

Description: Each Customer can place many Orders, and each Order belongs to exactly one Customer.
Min / Max:
Customer (1,N) → **Mandatory**
Order (1,1) → **Mandatory**
Participation: Both sides are Mandatory
Type of relationship: One-to-Many

2. **Employee – Processes – Order**:

Description: An Employee can process many Orders, and each Order is processed by exactly one Employee.
Min / Max:
Employee (1,N) → **Mandatory**
Order (1,1) → **Mandatory**
Participation: Both sides are Mandatory
Type of relationship: One-to-Many

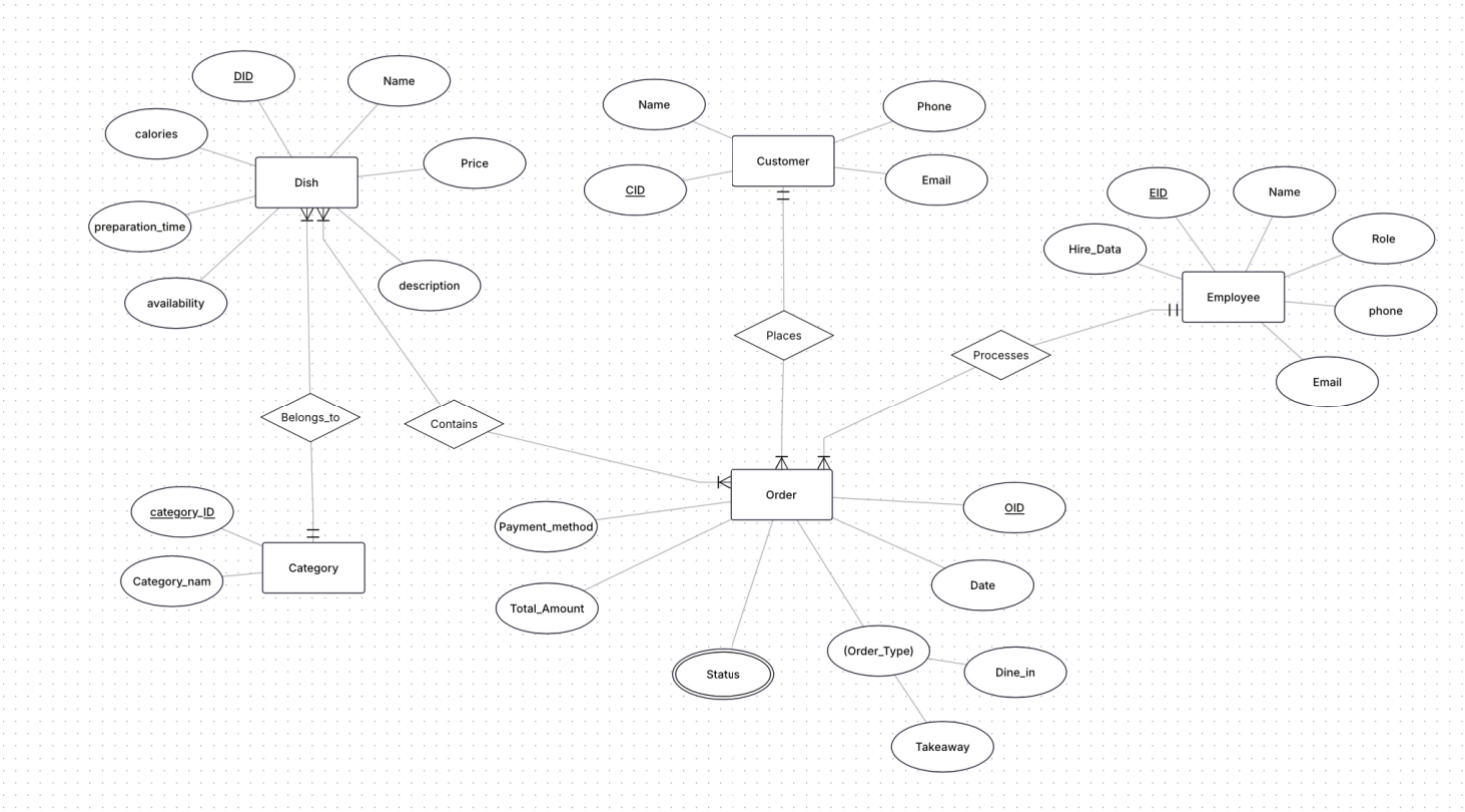
3. **Order – Contains – Dish:**

Description: Each Order must include at least one Dish, and each Dish must appear in at least one Order.
Min / Max:
Order (1,N) → **Mandatory**
Dish (1,N) → **Mandatory**
Participation: Order Mandatory, Dish Mandatory
Type of relationship: Many-to-Many

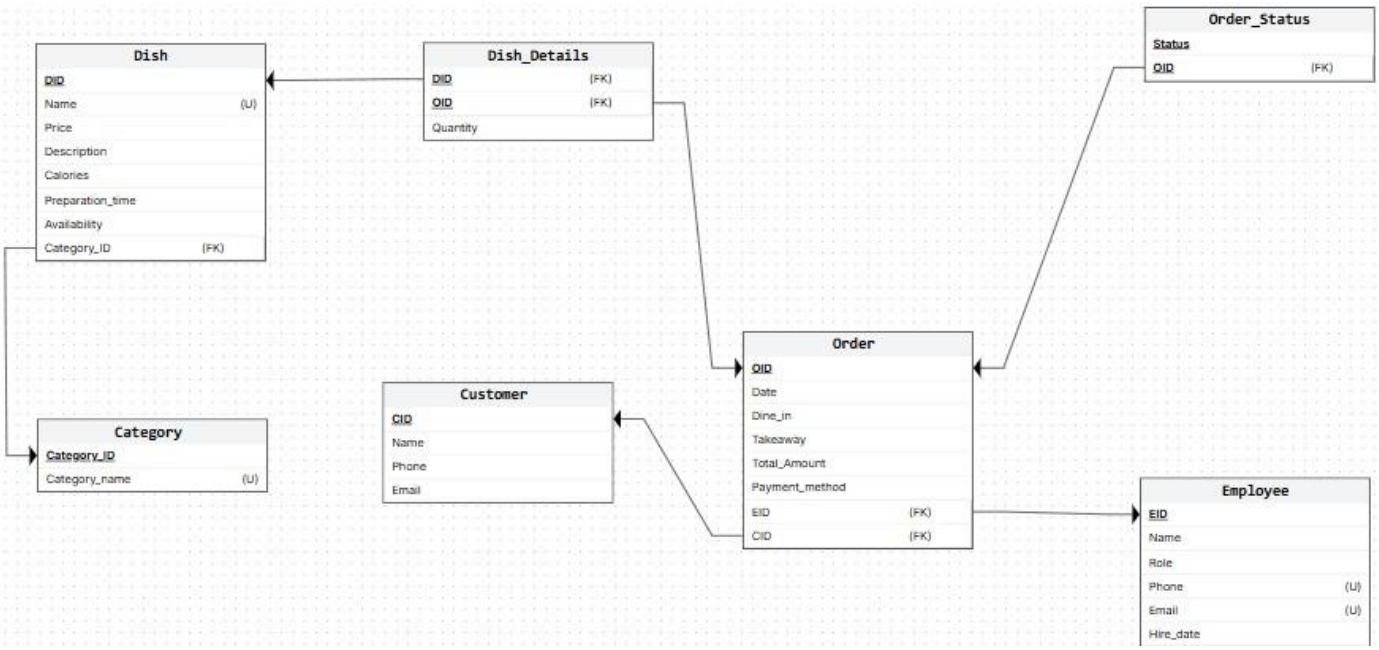
4. **Dish – Belongs_to – Category:**

Description: Each Dish belongs to exactly one Category, and each Category contains one or more Dishes.
Min / Max:
Dish (1,1) → **Mandatory**
Category (1,N) → **Mandatory**
Participation: Both sides are Mandatory
Type of relationship: One-to-Many

1.2 ER Diagram



1.3 Relational Schema



2. Implementation

2.1 Create Database

```
CREATE DATABASE Kababa1;
USE Kababa1;
```

2.2 Create Tables, apply constraints and describe

- This table stores the **main food categories** offered by the restaurant, such as Main Dishes, Side Dishes, Drinks, and Sauces.

```
CREATE TABLE Category (
    Category_ID INT PRIMARY KEY NOT NULL,
    Category_name ENUM('Main Dishes','Side Dishes','Drinks','Sauces') NOT NULL
);
describe Category;
```

Field	Type	Null	Key	Default	Extra
Category_ID	int	NO	PRI	NULL	
Category_name	enum('Main Dishes','Side Dishes','Drinks','Sauces')	NO		NULL	

- This table contains **detailed information about each dish** served at Kababa, including its name, price, description, availability, and nutritional details.

```sql
CREATE TABLE Dish (
    DID INT PRIMARY KEY NOT NULL,
    DName VARCHAR(100) NOT NULL,
    Price DECIMAL(6,2) NOT NULL,
    Description VARCHAR(150),
    Availability VARCHAR(20),
    Preparation_time INT,
    Calories INT,
    Category_ID INT,
    FOREIGN KEY (Category_ID) REFERENCES Category(Category_ID)
);

describe dish;
```

Field	Type	Null	Key	Default	Extra
DID	int	NO	PRI	NULL	
DName	varchar(100)	NO		NULL	
Price	decimal(6,2)	NO		NULL	
Description	varchar(150)	YES		NULL	
Availability	varchar(20)	YES		NULL	
Preparation_time	int	YES		NULL	
Calories	int	YES		NULL	
Category_ID	int	YES	MUL	NULL	

- This table records **information about the restaurant's employees**, including their name, role (Waiter, Cashier, or Chef), contact details, and hire date.

```sql
CREATE TABLE Employee(
    EID INT PRIMARY KEY NOT NULL,
    EName VARCHAR(100) NOT NULL,
    ERole ENUM('Waiter','Cashier','Chef'),
    EPhone VARCHAR(10) UNIQUE,
    EEmail VARCHAR(100) UNIQUE,
    EHire_date DATE
);

describe employee;
```

Field	Type	Null	Key	Default	Extra
EID	int	NO	PRI	NULL	
EName	varchar(100)	NO		NULL	
ERole	enum('Waiter','Cashier','Chef')	YES		NULL	
EPhone	varchar(10)	YES	UNI	NULL	
EEmail	varchar(100)	YES	UNI	NULL	
EHire_date	date	YES		NULL	

- This table stores **basic customer information** such as name, phone number, and email.

```sql
CREATE TABLE Custmer (
    CID INT PRIMARY KEY,
    Customer_Name VARCHAR(100),
    Customer_Phone VARCHAR(20),
    Customer_Email VARCHAR(100)
);
```

```sql
describe Customer;
```

Field	Type	Null	Key	Default	Extra
CID	int	NO	PRI	NULL	
Customer_Name	varchar(100)	YES		NULL	
Customer_Phone	varchar(20)	YES		NULL	
Customer_Email	varchar(100)	YES		NULL	

- This table contains **all customer orders**, including the order date, type (Dine-In or Takeaway), total amount, and payment method. **ON UPDATE CASCADE** updates related records automatically, while **ON DELETE SET NULL** preserves orders when a related record is deleted.

```sql
CREATE TABLE Orders (
    OID INT PRIMARY KEY,
    Dates DATETIME,
    Order_Type ENUM('Dine_In','Takeaway'),
    Total_Amount DECIMAL(8,2),
    Payment_Method ENUM('Cash','Card'),
    EID INT,
    CID INT,
    FOREIGN KEY (EID) REFERENCES Employee(EID)
    on delete set null
    on update cascade,
    FOREIGN KEY (CID) REFERENCES Customer(CID)
    on delete set null
    on update cascade
);
```

```sql
describe orders;
```

Field	Type	Null	Key	Default	Extra
OID	int	NO	PRI	NULL	
Dates	datetime	YES		NULL	
Order_Type	enum('Dine_In','Takeaway')	YES		NULL	
Total_Amount	decimal(8,2)	YES		NULL	
Payment_Method	enum('Cash','Card')	YES		NULL	
EID	int	YES	MUL	NULL	
CID	int	YES	MUL	NULL	

- This table records **the status of each order** (Pending, Preparing, Completed, or Cancelled) **as a multi-valued attribute.**

```
CREATE TABLE Orders_Status(
    Ostatus ENUM('pending','preparing','completed','cancelled'),
    OID INT,
    PRIMARY KEY (Ostatus, OID),
    FOREIGN KEY (OID) REFERENCES Orders(OID)
);
```

```
describe orders_status;
```

	Field	Type	Null	Key	Default	Extra
▶	Ostatus	enum('pending','preparing','completed','cancelled')	NO	PRI	NULL	
	OID	int	NO	PRI	NULL	

- This table represents the **many-to-many relationship between orders and dishes.** It stores which dishes are included in each order and the quantity of each dish.

```
CREATE TABLE Dish_Details (
    DID INT,
    OID INT,
    Quantity INT NOT NULL,
    PRIMARY KEY (DID, OID),
    FOREIGN KEY (DID) REFERENCES Dish(DID),
    FOREIGN KEY (OID) REFERENCES Orders(OID)
);
```

```
describe dish_details;
```

	Field	Type	Null	Key	Default	Extra
▶	DID	int	NO	PRI	NULL	
	OID	int	NO	PRI	NULL	
	Quantity	int	NO		NULL	

2.3 Insert Records

- Category

```sql
INSERT INTO Category (Category_ID, Category_name) VALUES
(1, 'Main Dishes'),
(2, 'Side Dishes'),
(3, 'Drinks'),
(4, 'Sauces');
```

Category_ID	Category_name
1	Main Dishes
2	Side Dishes
3	Drinks
4	Sauces

- Dish

```sql
INSERT INTO Dish
(DID, DName, Price, Description, Availability, Preparation_time, Calories, Category_ID)
VALUES
(1, 'Beef Kebab', 22.00, 'Grilled beef kebab', 'available', 20, 450, 1),
(2, 'Chicken Kebab', 20.00, 'Grilled chicken kebab', 'available', 18, 400, 1),
(3, 'French Fries', 8.00, 'Crispy fried potatoes', 'available', 7, 300, 2),
(4, 'Orange Juice', 6.00, 'Fresh orange juice', 'available', 4, 110, 3),
(5, 'Tahini', 3.00, 'Creamy tahini sauce', 'available', 2, 120, 4),
(6, 'Tawook', 13.00, 'Chicken sandwich with tawook marinade', 'available', 22, 374, 1),
(7, 'Fatosha', 19.00, 'Fresh Fatosha salad with crusty bread', 'available', 5, 42, 2),
(8, 'Lemon Juice', 10.00, 'Fresh lemon drink', 'available', 3, 114, 3),
(9, 'Garlic Sauce', 2.00, 'Creamy garlic dip', 'available', 1, 168, 4),
(10, 'Water', 1.00, 'Bottled drinking water', 'available', 1, 0, 3);
```

DID	DName	Price	Description	Availability	Preparation_time	Calories	Category_ID
10	Water	1.00	Bottled drinking water	available	1	0	3
6	Tawook	13.00	Chicken sandwich with tawook marinade	available	22	374	1
9	Garlic Sauce	2.00	Creamy garlic dip	available	1	168	4
5	Tahini	3.00	Creamy tahini sauce	available	2	120	4
3	French Fries	8.00	Crispy fried potatoes	available	7	300	2
7	Fatosha	19.00	Fresh Fatosha salad with crusty bread	available	5	42	2
8	Lemon Juice	10.00	Fresh lemon drink	available	3	114	3
4	Orange Juice	6.00	Fresh orange juice	available	4	110	3
1	Beef Kebab	22.00	Grilled beef kebab	available	20	450	1

- Employee

```sql
INSERT INTO Employee (EID, EName, ERole, EPhone, EEmail, EHire_date) VALUES
(1, 'Ahmed Mohamed', 'Waiter', '0501234567', 'ahmed.mohamed@example.com', '2023-01-10'),
(2, 'Sara Ali', 'Cashier', '0502345678', 'sara.ali@example.com', '2023-03-15'),
(3, 'Khaled Abdullah', 'Chef', '0503456789', 'khaled.abdullah@example.com', '2023-05-20'),
(4, 'Noura Salem', 'Chef', '0504567890', 'noura.salem@example.com', '2023-07-01'),
(5, 'Fahd Hussein', 'Chef', '0505678901', 'fahd.hussein@example.com', '2023-08-12'),
(6, 'Laila Ahmed', 'Waiter', '0506789012', 'laila.ahmed@example.com', '2023-09-18'),
(7, 'Mohammed Saad', 'Cashier', '0507890123', 'moh.saad@example.com', '2023-10-05'),
(8, 'Hussain Omar', 'Chef', '0508901234', 'hussain.omar@example.com', '2023-11-02'),
(9, 'Rana Ibrahim', 'Waiter', '0509012345', 'rana.ibrahim@example.com', '2024-01-12'),
(10, 'Salem Yaser', 'Chef', '0501122334', 'salem.yaser@example.com', '2024-02-20');
```

EID	EName	ERole	EPhone	EEmail	EHire_date
1	Ahmed Mohamed	Waiter	0501234567	ahmed.mohamed@example.com	2023-01-10
2	Sara Ali	Cashier	0502345678	sara.ali@example.com	2023-03-15
3	Khaled Abdullah	Chef	0503456789	khaled.abdullah@example.com	2023-05-20
4	Noura Salem	Chef	0504567890	noura.salem@example.com	2023-07-01
5	Fahd Hussein	Chef	0505678901	fahd.hussein@example.com	2023-08-12
6	Laila Ahmed	Waiter	0506789012	laila.ahmed@example.com	2023-09-18
7	Mohammed Saad	Cashier	0507890123	moh.saad@example.com	2023-10-05
8	Hussain Omar	Chef	0508901234	hussain.omar@example.com	2023-11-02
9	Rana Ibrahim	Waiter	0509012345	rana.ibrahim@example.com	2024-01-12

- Customer

```sql
INSERT INTO Customer (CID, Customer_Name, Customer_Phone, Customer_Email) VALUES
(1, 'Ali Hassan', '0551234567', 'ali.hassan@example.com'),
(2, 'Mona Saleh', '0552345678', 'mona.saleh@example.com'),
(3, 'Yousef Ahmad', '0553456789', 'yousef.ahmad@example.com'),
(4, 'Reem Abdullah', '0554567890', 'reem.abdullah@example.com'),
(5, 'Omar Khalid', '0555678901', 'omar.khalid@example.com'),
(6, 'Huda Majed', '0556789012', 'huda.majed@example.com'),
(7, 'Salman Adel', '0557890123', 'salman.adel@example.com'),
(8, 'Dalia Hassan', '0558901234', 'dalia.hassan@example.com'),
(9, 'Majid Younis', '0559012345', 'majid.younis@example.com'),
(10, 'Fatimah Saeed', '0550123456', 'fatimah.saeed@example.com');
```

CID	Customer_Name	Customer_Phone	Customer_Email
1	Ali Hassan	0551234567	ali.hassan@example.com
2	Mona Saleh	0552345678	mona.saleh@example.com
3	Yousef Ahmad	0553456789	yousef.ahmad@example.com
4	Reem Abdullah	0554567890	reem.abdullah@example.com
5	Omar Khalid	0555678901	omar.khalid@example.com
6	Huda Majed	0556789012	huda.majed@example.com
7	Salman Adel	0557890123	salman.adel@example.com
8	Dalia Hassan	0558901234	dalia.hassan@example.com
9	Majid Younis	0559012345	majid.younis@example.com

- Orders

```sql
INSERT INTO Orders (OID, Dates, Order_Type, Total_Amount, Payment_Method, EID, CID) VALUES
(1, '2025-01-10 14:20:00', 'Dine_In', 64.00, 'Cash', 1, 1),
(2, '2025-01-11 15:45:00', 'Takeaway', 36.00, 'Card', 2, 2),
(3, '2025-01-11 18:10:00', 'Dine_In', 6.00, 'Cash', 1, 3),
(4, '2025-01-12 12:30:00', 'Takeaway', 3.00, 'Card', 2, 4),
(5, '2025-01-12 19:00:00', 'Dine_In', 42.00, 'Cash', 3, 5),
(6, '2025-01-13 13:10:00', 'Dine_In', 55.00, 'Cash', 6, 6),
(7, '2025-01-13 16:45:00', 'Takeaway', 27.00, 'Card', 7, 7),
(8, '2025-01-14 11:30:00', 'Dine_In', 38.00, 'Cash', 8, 8),
(9, '2025-01-14 14:55:00', 'Takeaway', 22.00, 'Card', 9, 9),
(10, '2025-01-15 18:00:00', 'Dine_In', 70.00, 'Cash', 10, 10);
```

OID	Dates	Order_Type	Total_Amount	Payment_Method	EID	CID
1	2025-01-10 14:20:00	Dine_In	64.00	Cash	1	1
2	2025-01-11 15:45:00	Takeaway	36.00	Card	2	2
3	2025-01-11 18:10:00	Dine_In	6.00	Cash	1	3
4	2025-01-12 12:30:00	Takeaway	3.00	Card	2	4
5	2025-01-12 19:00:00	Dine_In	42.00	Cash	3	5
6	2025-01-13 13:10:00	Dine_In	55.00	Cash	6	6
7	2025-01-13 16:45:00	Takeaway	27.00	Card	7	7
8	2025-01-14 11:30:00	Dine_In	38.00	Cash	8	8
9	2025-01-14 14:55:00	Takeaway	22.00	Card	9	9

- Orders_Status

```
INSERT INTO Orders_Status (Ostatus, OID) VALUES
('pending', 1),
('completed', 2),
('cancelled', 3),
('completed', 4),
('preparing', 5),
('preparing', 6),
('completed', 7),
('pending', 8),
('preparing', 9),
('completed', 10);
```

Ostatus	OID
pending	1
pending	8
preparing	5
preparing	6
preparing	9
completed	2
completed	4
completed	7
completed	10

- Dish_Details

```
INSERT INTO Dish_Details (DID, OID, Quantity) VALUES
(1, 1, 2),
(2, 1, 1),
(3, 2, 2),
(4, 3, 1),
(5, 4, 1),
(6, 6, 2),
(7, 7, 1),
(8, 8, 3),
(9, 9, 1),
(10, 10, 2);
```

DID	OID	Quantity
1	1	2
2	1	1
3	2	2
4	3	1
5	4	1
6	6	2
7	7	1
8	8	3
9	9	1

2.4 Basic Select

- selects customer names and phone numbers where the phone number starts with '0553' **Dana**

```
SELECT Customer_Name, Customer_Phone
FROM Customer
WHERE Customer_Phone LIKE '0553%';
```

Customer_Name	Customer_Phone
Yousef Ahmad	0553456789

- Displays all employee names who work as Chefs. **Dana**

```
SELECT EName AS Chef_Employee
FROM Employee
WHERE ERole = 'Chef';
```

Chef_Employee
Khaled Abdullah
Noura Salem
Fahd Hussein
Hussain Omar
Salem Yaser

- Shows dishes that have a price higher than 10 SAR. **Hajer**

```
SELECT DName, Price FROM Dish WHERE Price > 10;
```

DName	Price
Beef Kebab	22.00
Chicken Kebab	20.00
Tawook	13.00
Fatosha	19.00
Orange Juice	12.00

- It displays dishes that have fewer than 100 calories. **Hajer**

```
SELECT DName, Calories FROM Dish WHERE Calories < 100;
```

DName	Calories
Fatosha	42
Water	0

- Displays available dishes. **Alaa**

```
SELECT DName, Price
FROM Dish
WHERE Availability = 'available';
```

DName	Price
Beef Kebab	22.00
Chicken Kebab	20.00
French Fries	8.00
Orange Juice	6.00
Tahini	3.00
Tawook	13.00
Fatosha	19.00
Lemon Juice	10.00
Garlic Sauce	2.00
Water	1.00

- Lists customers whose phone number ends with the digit 9. **Alaa**

```
SELECT Customer_Name, Customer_Phone
FROM Customer
WHERE Customer_Phone LIKE '%9';
```

Customer_Name	Customer_Phone
Yousef Ahmad	0553456789

2.5 Aggregate functions

- Calculates the total revenue from all orders. **Dana**

```
SELECT SUM(Total_Amount) AS Total_Revenue
FROM Orders;
```

Total_Revenue
363.00

- Finds the highest dish price on the menu. **Dana**

```
SELECT MAX(Price) AS Highest_Price
FROM Dish;
```

Highest_Price
22.00

- Counts the total number of orders in the system. **Hajer**

```
SELECT COUNT(*) FROM Orders;
```

COUNT(*)
10

- Calculate the average price of dishes for each category. **Hajer**

```
SELECT Category_ID, AVG(Price)
FROM Dish GROUP BY Category_ID;
```

Category_ID	AVG(Price)
1	18.333333
2	13.500000
3	9.333333
4	2.500000

- Counts the total number of dishes available in the menu. **Alaa**

```
SELECT COUNT(*) AS Total_Dishes
FROM Dish;
```

Total_Dishes
10

- Finds the highest total amount among all recorded orders. **Alaa**

```
SELECT MAX(Total_Amount) AS Highest_Order_Amount
FROM Orders;
```

Highest_Order_Amount
70.00

2.6 Join Between Tables

- Lists all orders along with their status. **Dana**

```
SELECT Orders.OID, Orders_Status.Ostatus
FROM Orders
JOIN Orders_Status ON Orders.OID = Orders_Status.OID;
```

OID	Ostatus
1	pending
8	pending
5	preparing
6	preparing
9	preparing
2	completed
4	completed
7	completed
10	completed
3	cancelled

- Retrieves customers and the total amount they spent in every order.

Dana

```
SELECT Customer.Customer_Name, Orders.Total_Amount
FROM Customer
JOIN Orders ON Customer.CID = Orders.CID;
```

Customer_Name	Total_Amount
Ali Hassan	64.00
Mona Saleh	36.00
Yousef Ahmad	6.00
Reem Abdullah	3.00
Omar Khalid	42.00
Huda Majed	55.00
Salman Adel	27.00
Dalia Hassan	38.00
Majid Younis	22.00
Fatimah Saeed	70.00

- displays the order number, customer name, and payment method.

Hajer

```
-- displays the order number, customer name, and payment method.
SELECT Orders.OID, Customer.Customer_Name, Orders.Payment_Method
FROM Orders
JOIN Customer ON Orders.CID = Customer.CID;
```

OID	Customer_Name	Payment_Method
1	Ali Hassan	Cash
2	Mona Saleh	Card
3	Yousef Ahmad	Cash
4	Reem Abdullah	Card
5	Omar Khalid	Cash
6	Huda Majed	Cash
7	Salman Adel	Card
8	Dalia Hassan	Cash
9	Majid Younis	Card
10	Fatimah Saeed	Cash

- Each order number displays its dish name and quantity. **Hajer**

```
SELECT Orders.OID, Dish.DName, dish_details.Quantity
FROM Orders
JOIN dish_details ON Orders.OID = dish_details.OID
JOIN Dish ON Dish.DID = dish_details.DID;
```

OID	DName	Quantity
1	Beef Kebab	2
1	Chicken Kebab	1
3	Orange Juice	1
2	French Fries	2
4	Tahini	1
6	Tawook	2
7	Fatosha	1
8	Lemon Juice	3
9	Garlic Sauce	1
10	Orange Juice	2

- Shows customer names along with their corresponding order IDs. **Alaa**

```sql
SELECT Customer.Customer_Name, Orders.OID
FROM Customer
JOIN Orders ON Customer.CID = Orders.CID;
```

Customer_Name	OID
Ali Hassan	1
Mona Saleh	2
Yousef Ahmad	3
Reem Abdullah	4
Omar Khalid	5
Huda Majed	6
Salman Adel	7
Dalia Hassan	8
Majid Younis	9
Fatimah Saeed	10

- Shows dish names along with the category each dish belongs to. **Alaa**

```sql
SELECT Dish.DName, Category.Category_name
FROM Dish
JOIN Category ON Dish.Category_ID = Category.Category_ID;
```

DName	Category_name
Beef Kebab	Main Dishes
Chicken Kebab	Main Dishes
Tawook	Main Dishes
French Fries	Side Dishes
Fatosha	Side Dishes
Orange Juice	Drinks
Lemon Juice	Drinks
Orange Juice	Drinks
Tahini	Sauces
Garlic Sauce	Sauces

2.7 Subqueries to Answer Specific Question

- Find dishes that have more calories than the average calories in their category. **Dana**

```
SELECT DName, Calories
FROM Dish d
WHERE Calories > (
    SELECT AVG(Calories)
    FROM Dish
    WHERE Category_ID = d.Category_ID
);
```

DName	Calories
Beef Kebab	450
French Fries	300
Garlic Sauce	168
Orange Juice	133

- Find dishes that were **never ordered**. **Dana**

```
SELECT DName
FROM Dish
WHERE DID NOT IN (
    SELECT DISTINCT DID
    FROM Dish_Details
);
```

DName

- It display dishes that are more expensive compared to the rest of the menu. **Hajer**

```
SELECT DName
FROM Dish
WHERE Price > (SELECT AVG(Price) FROM Dish);
```

DName
Beef Kebab
Chicken Kebab
Tawook
Fatosha
Orange Juice

- It display Customers whose total orders are greater than 50. **Hajer**

```sql
SELECT Customer_Name
FROM Customer
WHERE CID IN (
    SELECT CID FROM Orders WHERE Total_Amount > 50
);
```

Customer_Name
Ali Hassan
Huda Majed
Fatimah Saeed

- Displays dishes that require more preparation time than the average. **Alaa**

```sql
SELECT DName
FROM Dish
WHERE Preparation_time > (SELECT AVG(Preparation_time) FROM Dish);
```

DName
Beef Kebab
Chicken Kebab
Tawook

- Shows the dishes that belong to the "Drinks" category. **Alaa**

```sql
SELECT DName, Price
FROM Dish
WHERE Category_ID = (
    SELECT Category_ID
    FROM Category
    WHERE Category_name = 'Drinks'
);
```

DName	Price
Orange Juice	6.00
Lemon Juice	10.00
Water	1.00

2.8 View

- View to show average price and calories for each category. **Dana**

```
CREATE VIEW Category_Stats AS
SELECT Category.Category_name, AVG(Dish.Price) AS Avg_Price, AVG(Dish.Calories) AS Avg_Calories
FROM Category
JOIN Dish ON Category.Category_ID = Dish.Category_ID
GROUP BY Category.Category_name;
```

Category_name	Avg_Price	Avg_Calories
Main Dishes	18.333333	408.0000
Side Dishes	13.500000	171.0000
Drinks	9.333333	119.0000
Sauces	2.500000	144.0000

- View to show each order with the customer's name and total amount. **Hajer**

```
CREATE VIEW Order_Report AS
SELECT Orders.OID, Customer.Customer_Name AS Customer_Name, Orders.Total_Amount
FROM Orders
JOIN Customer ON Orders.CID = Customer.CID;

select * from Order_Report;
```

OID	Customer_Name	Total_Amount
1	Ali Hassan	64.00
2	Mona Saleh	36.00
3	Yousef Ahmad	6.00
4	Reem Abdullah	3.00
5	Omar Khalid	42.00
6	Huda Majed	55.00
7	Salman Adel	27.00
8	Dalia Hassan	38.00
9	Majid Younis	22.00
10	Fatimah Saeed	70.00

- Creates a view that shows dish names and their prices. **Alaa**

```
CREATE VIEW Dish_Prices_View AS
SELECT DName, Price
FROM Dish;
```

```
SELECT * FROM Dish_Prices_View;
```

DName	Price
Beef Kebab	22.00
Chicken Kebab	20.00
French Fries	8.00
Orange Juice	6.00
Tahini	3.00
Tawook	13.00
Fatosha	19.00
Lemon Juice	10.00
Garlic Sauce	2.00
Water	1.00

جامعة أم القرى

UMM AL-QURA UNIVERSITY